

Chapter Name

Python Fundamentals

Index:

1. Python Character Set
2. Tokens:
 - 2.1 Keywords
 - 2.2 Identifiers (Names)
 - 2.3 Literals
 - 2.4 Operators
 - 2.5 Punctuators
3. Barebones of Python Program
4. Variables and Assignments
5. Simple Input & Output
6. Output & Error Practice Questions

1. Python Character Set

Character set is a set of valid characters that a language can recognize.

Python has the following character set:

1. Letters - A(65)-Z(90) , a(97)-z(122)
2. Digits - 0(48) - 9(57)
3. Special Symbols - space , + - etc
4. Whitespace - Blank Space, tabs , newline

2. Tokens/lexical unit

The smallest individual unit in a program is known as a Token or a lexical unit.

An English Sentence

In []:

```
1 "Hurray!", he exclaimed, "We are the world champions."
```

A Python Program

In []:

```
1 # A sample program
2 name = 'Anya'
3 age = 15
4 contribution = 710.75
5 print(name, "aged", age, 'has contributed', contribution)
```

2.1 Keywords

A keyword is a word having special meaning reserved by programming language.

Note:- Most of the keywords will be available in smallcase only

example

In []:

```
1 True, False, for, in, not, not in, if, else, elif, etc.
```

2.2 Identifier(Names)

Identifier are fundamental building blocks of a program and are used as the general terminology for the names given to different parts of the program such as - variables, objects, classes, functions, name, lists, dictionaries, string, tuple, etc.

Identifier Forming Rules of Python

In []:

```
1 1. The first character must be letter or underscore.
2 2. Identifier must not contain any space.
3 3. Uppercase and lowercase identifier considered differently by Python lang.
4 4. Digits only allowed in between/at last the identifier.
5 5. No special characters allowed in identifier except UNDERSCORE.
6 6. Identifier must not be a keyword of Python.
```

Example - V- Valid & IV - Invalid

In []:

```

1 Myfile      - V
2 Data-Rec    - IV
3 29Kohli     - IV
4 In          - V
5 in          - IV
6 My.life     - IV
7 File420     - V
8 _CS         - V
9 _12         - V
10 DOB_08_07_21 - V
11 &name       - IV
12 Import     - V

```

2.3 Literals / Data / Values / Constants

Literals are data items that have a fixed value.

Python allows several kinds of literals:

- 2.3.1. Numeric
- 2.3.2. String
- 2.3.3. Boolean
- 2.3.4. Special Literals - None

2.3.1. Numeric Literals

This has divided into 3 different types:

- a. int(signed integers)
- b. float
- c. Complex

a. int(signed integers):

- All +ve and -ve whole numbers without decimal point.
- example - 125 , 420 , 840, -450, -960

Python allows three types of integer literals:

- a.a - Decimal Integer Literals
- a.b - Octal Integer Literals
- a.c - Hexadecimal Integer Literals

a.a Decimal Integer:

- It consist total 10 digits
- range-> 0 to 9
- Example - 155,859,656

a.b Octal Integer:

- A sequence of digits starting with 0o(digit zero followed by letter o) is taken to be an Octal Integer
- It consist only 8 digits
- range - > 0 to 7
- Example - 0o28(IV) , 0o987(IV) , 0o777 ,0o762

a.c Hexadecimal Integer:

- A sequence of digits preceded by 0x or 0X is taken to be an hexadecimal integer
- It consist only 16 digits
- range - 0 to 9 (10-A , 11-B, 12-C ,13-D ,14-E ,15-F)
- Example - 0XBA9 , 0xBK9 , 0x19AZ

b. float - Numbers with decimal point

example - 3.14 , 2.75

Python supports two Forms of float literals:

- b.a Fractional Form
- b.b Exponent Form

b.a Fractional Form:

- Example - 2.0,17.5,-13.0,-0.00256

b.b Exponent Form:

- A real constant in exponent form has two part:
 - mantissa
 - exponent
- The mantissa must be either an integer or a proper real constant
- E or e is used to represent exponent part of any Exponent form of float literal

In []:

```

1 123 * 10power4 - 123E04
2 0.58 * 10power1 - 0.58E01
3 0.58 * 10power5 - 0.58E05
4 0.58 * 10power-5 - 0.58E-5

```

c. Complex - this usually will available in form of a+bi (i-imaginary)

example - 12 + 2i , 12.5 +2.6j

2.3.3. Boolean Literals

A boolean literal in Python is used to represent one of the two Boolean values

example - True or False

2.3.2. String

A string literal is a sequence of characters surrounded by quotes (single/double/triple).

In []:

```

1 s1 = 'i love India'
2 s2 = "i love India"
3 s3 = '''i love India'''
4 print(s1,s2,s3)
5 s4 = 'Aman420+--*'
6 print(s4)

```

Nongraphic Characters

1.Nongraphic Characters are those characters that cannot be typed directly from keyboard such as backspace,tabs,etc.(No Character is typed when these keys are pressed , only some action takes place)

2. These non graphic characters can be represented by using ESCAPE SEQUENCES

Escape Sequence:

- An escape sequence is represented by a backslash(\) followed by one or more characters

\n - Newline Character \t - Tab \ - Backslash \' - Single Quote \" - Double Quote \b - ASCII Backspace

In []:

```
1 #print("aman\nar\nyan")
2 #print("one\t\t\ttwo")
3 #print("aman's car")
4 #print('aman"aryan')
5 print("a\bman" )
```

In []:

```
1 s='am\'an'
2 print(len(s))
```

String Types in Python:

1. Single-line Strings(Default/Basic strings)
2. Multiline Strings

Single-line Strings(Default/Basic strings)

The string that we create by enclosing text in `"/"` are normally single line string.(terminate in one line)

For Example

In []:

```
1 s = 'i am jupyter notebook'
2 print(s)
```

Multiline Strings

Text or string spread across multiple lines instead written in one line is called multiline string.

Multiline strings can be created in two ways:

- 1.By adding a backslash at the end of normal `"/"` strings.
- 2.By typing the text in triple quotation marks(No backslash required)

1.By adding a backslash at the end of normal `"/"` strings.

Example:

In []:

```
1 s = "i am spiderman and i can swim asd lkjad lajsdlk lj \  
2 ljsd lasjdlkas dlkasjdlka sdlkjaslkd jalksd lkasjd \  
3 lkasjdlkasjdlk ajkasjd"  
4 print(s)
```

2.By typing the text in triple quotation marks(No backslash required)

In []:

```
1 s='''i am spiderman and i can swim asd  
2 lkjad lajsdlk ljasldk  
3 lj lkasjdlk ljsd lasjdlka  
4  
5 s dlkasjdlka sdlkjaslkd jalksd lkasjd  
6 lkasjdlkasjdlk ajkasjd'''  
7 print(s)
```

In []:

```
1 s1="aman \  
2 aryan"  
3 print(s1)  
4 print(len(s1))
```

In []:

```
1 s2='''aman  
2 aryan  
3  
4 '''  
5 print(s2)  
6 print(len(s2))
```

2.3.4. Special Literal - None

The None literal is used to indicate absence of value.

In []:

```
1 Shrey = None  
2 print(Shrey)
```

2.4 Operators

Operators are tokens that trigger some computation/action when applied to variables and other objects in an expression.

Operands

Object on which some kind of computation or action performed by any operator is called OPERANDS

In []:

```
1 b=10
2 c=30
3 10 + b - 30 * c / 2
```

Two different category of Operators are:

1. Unary Operator
2. Binary Operator

1. Unary Operator:

Unary operator are those operators that require one operand to operate upon.

Following are some unary operators:

- Unary plus(+)
- Unary minus(-)
- Logical negation (not)

2.Binary Operator

Binary operators are those operators that require two operands to operate upon.

Following are some Binary Operators:

1. Arithmetic
2. Relational
3. Logical Opt
4. Assignment Opt
5. Membership Opt
6. Identity Opt

1.Arithmetic Operator

+(addition)

-(Subtraction)

*(Multiplication)

/(division)

%(Modulus/Remainder)

******(Exponent(raise to power))

//(Floor Division)

2. Relational Operators

<(less than)

(greater than)

<=(less than equal to)

=(greater than equal to)

==(Equal to)

!=(Not equal to)

3. Logical Operator

and (Logical AND Operator)

or (Logical OR Operator)

4. Assignment Operator

1. **=** assignment
2. **/=** Assign Quotient
3. **+=** Assign Sum
4. ***=** Assign Product
5. **%=** Assign remainder
6. **-=** Assign difference
7. ****=** Assign Exponent
8. **//=** Assign Floor Division

5. Membership Operators

1. **in** (Whether variable in sequence)
2. **not in**(Whether variable not in sequence)

6. Identity Operator

In []:

```

1 1. is (is the identity same?)
2 2. is not(is the identity not same?)

```

2.5. Punctuators

Punctuators are symbols that are used in programming languages to organize sentence structure, and indicate the rhythm and emphasis of expressions, statements and program structure.

Example of Punctuators:

```

' '
" "
#
\
()
[]
{}
@
,
:
.

```

In []:

```

1 (10+20)-30/2*100

```

3. Barebones of a Python Program

In []:

```

1 '''
2     This is multiline Comments
3
4
5 '''
6 #This program shows a program's components
7 #Definition of function SeeYou() follows
8 def SeeYou():
9     print("Time to say Good Bye!!")
10 #Main Program-code follows now
11 a=15
12 b=a-10
13 print(a+3)
14 if b>5:                # Colon means it's block
15     print("Value of 'a' was more than 15 initially")
16 else:
17     print("Value of 'a' was 15 or less initially")
18 SeeYou()                #Calling above defined function SeeYou()

```

Above program contains various following components:

1. expressions
2. statements
3. comments
4. function
5. blocks and indentation

3.Comments

Comments are the additional readable information to clarify the source code.

Two forms of comments are:

1. Single Line comments
 - a. Full line comments
 - b. Inline Comments
2. Multiline Comments

a. Full Line Comments:

- The physical lines beginning with # are the full line comments(See Line no 6 & 7)

b. Inline Comments:

- It starts in the middle of a physical line, after Python code(See Line 14 & 18)

In []:

```
1 2. Multiline Comments:(docstrings)
2  -
```

1.Expressions

1. An expression is any legal combination of symbols that represents a value.
2. Which Python Evaluates.
3. Produces a value

Example:

In []:

```
1 15
2 2.9
3 a+5
4 (3+5)/4
```

2. Statement

1. A statement is a programming instruction that does something i.e., some action takes place.
2. A statement executes and may or may not yield a value.

Example:

In []:

```
1 print("Hello i m RAIN")
```

In []:

```
1 a=10
2 if a>=10:
3     print("I m 10")
4 else:
5     print("I m not 10")
```

4. Blocks & Indentation

1. A group of statements which are part of another statement or a function are called block or code-block or suite in Python.
2. Python uses indentation to create block of code.
3. Statements at same indentation level are part of same block/suite.
4. Statements requiring suite/code-block have a colon(:) at their end.
5. You cannot unnecessarily indent a statement; Python will raise error for that.

ROOT/MAIN/GLOBAL LEVEL AREA/BLOCK

In []:

```
1 print("hello")
2 print("bye")
3     a=10           #It is with one Indent Space
4     b=20
5 c=a+b
```

In Python Language

In []:

```

1  a=11
2  if a==10:
3      print("I am 10")
4      print("bye")           #It is If Block
5      print("ok")
6  else:
7      print("I m not 10")    #IT is Else Block
8      print("see")

```

In C++ language

In []:

```

1  int a=10
2  if(a==10)
3  {
4      print("sdsds")
5      print("sdsds")
6      print("sdsds")
7  }
8  else
9  {
10 print("sdsds")
11 print("sdsds")
12 }

```

5. Function

1. A function is a code that has a name and it can be reused (executed again) by specifying its name in the program, where needed.
2. To create function we need to use Keyword "def"
3. Each function usually contains Body of Function.
4. Every function can be used once it is called by programmer.

Format/Syntax of Function

In []:

```

1  #Function Definition = Function Header + Function Body
2  def <Functionname>([optional arguments]):    # Function Header
3      .....<Function Body>                    #Function Body
4      .....<Function Body>

```

In []:

```

1  #Function Calling Code
2  <FunctionName>(argument)

```

In []:

```
1 a=10
2 b=20
3 c=a+b
4 print(c)
```

In []:

```
1 #Function Definition
2 def ADD(a,b):
3     print(a+b)
4
5 def Double(n):
6     print(n*n)
7
8
9 #Function Calling Code
10 ADD(100,200)
11 Double(200)
```

4.Variables and Assignments

1. Named labels, whose values can be used and processed during program run,are called Variables.
2. Variable also called Symbolic Variables.

How to create/Define variable

Format of creating variable:

<variablename> = <value>

In []:

```
1 a=10
2 b=10
3 c='aman'
4 h='aman'
```

In []:

```
1 print(id(10),"<-",id(a))
2 print(id(10),"<-",id(b))
3 print(id('aman'),'<-',id(c))
4 print(id(h))
```

Lvalues and Rvalues

Lvalues:

- Lvalues are the objects to which you can assign a value or expression .
- Lvalues can come on lhs or rhs of an assignment statement.

In []:

```
1 A=10
2 A=10+20
3 B=A
```

Rvalues:

- Rvalues are the literals and expressions that are assigned to lvalues.
- Rvalues can come on rhs of an assignment statements.

In []:

```
1 a=10
2 b=3.14
3 c='aman'
4 d=10+20*20-30
```

Multiple Assignments

1. Assigning some value to multiple variables
2. Assigning multiple values to multiple variables

1. Assigning same value to multiple variables

In []:

```
1 a=10
2 b=10
3 c=10
```

or

In []:

```
1 a=b=c=10
```

2. Assigning multiple values to multiple variables

In []:

```
1 a=10
2 b=20
3 c=30
```

OR

In []:

```
1 a,b,c = 10,20,30
```

In [1]:

```
1 a,b,c = 10,20,30
2 s1,s2,s3 = 'aman','aryan','amit'
3 print(a,b,c)
4 print(s1,s2,s3)
```

```
10 20 30
aman aryan amit
```

In [2]:

```
1 x,y=25,50
2 print(x,y)
```

```
25 50
```

This code performs INTERCHAGE of values between two variables

In [4]:

```
1 x,y=10,20
2 x,y = y,x      #20  10
3 print(x,y)
```

```
20 10
```

In [5]:

```
1 a,b,c=5,10,7
2 b,c,a = a+1,b+2,c-1    #6    12    6
3 print(a,b,c)
```

```
6 6 12
```

In [6]:

```
1 p,q = 3,5
2 q,r = p-2 , p+2    #1    5
3 print(p,q,r)
```

```
3 1 5
```

In [7]:

```
1 x,x=20,30
2 y,y = x+10,x+20    #30+10=40 ,  30+20=50
3 print(x,y)
```

```
30 50
```

In [8]:

```
1 x=10
2 y,y = x+2,x+5      #12 , 15
3 print(x,y)
```

10 15

Variable Definition

In [9]:

```
1 z
2 print(z)
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-9-6136f9757023> in <module>()
----> 1 z
      2 print(z)
```

NameError: name 'z' is not defined

In [10]:

```
1 z=100
2 print(z)
```

100

Dynamic Typing

A variable pointing to a value of a certain type, can be made of point to a value/object of different type. This is called Dynamic Typing

In [1]:

```
1 a=10
2 print(a)
3 a=3.14
4 print(a)
5 a='aman'
6 print(a)
```

10
3.14
aman

Static Typing

In Static typing , a data type is attached/associated with a variable when it is defined first and it is fixed.

In C++ language

In []:

```
1 int a=10
2 print(a)
3 a='aman'
4 print(a)
```

Drawback of Dynamic Typing

In [2]:

```
1 X=10
2 Y=0
3 Y=X/2
4 print(X,Y)
```

10 5.0

In [4]:

```
1 X='Day'
2 print(type(X))
3 Y=X/2
4 print(X,Y)
```

<class 'str'>

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-4-d7b7c1b37d5c> in <module>()
      1 X='Day'
      2 print(type(X))
----> 3 Y=X/2
      4 print(X,Y)
```

TypeError: unsupported operand type(s) for /: 'str' and 'int'

type() function

1. Use type(object name) function to determine the type of an object.
2. Object can be a variable or a literal etc.

In [5]:

```
1 a=10
2 b=10.5
3 c='aman'
```

In [6]:

```
1 print(type(a))
2 print(type(b))
3 print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

In [7]:

```
1 print(type(10))
2 print(type(10.5))
3 print(type('aman'))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

5.Simple Input and Output(print) Function

Input Function

1. It is built-in function of python lang
2. It is used to accept any value from user in run-time
3. The input() function always returns a value of string type.

Format/Syntax of input() function

1st format

In []:

```
1 variablename = input(<prompt to be displayed>)
```

2nd Format

In []:

```
1 variablename = int/float/str(input(<Promt to b displayed>))
2
3 int()
4 float()
5 str()
6 It is a function to convert value into either of this type int/float/string.
```

In [8]:

```
1 name = input("Enter Your Name:- ")
2 print(name, " ", type(name))
```

Enter Your Name:- aryan
 aryan <class 'str'>

In [9]:

```
1 age = input("Enter Your Age:- ")
2 print(age, " ", type(age))
```

Enter Your Age:- 17
 17 <class 'str'>

In [10]:

```
1 a=input("Enter value of a:- ")
2 b=input("Enter value of b:- ")
3 c=a+b
4 print(c, " ", type(a), " ", type(b), " ", type(c))
```

Enter value of a:- 100
 Enter value of b:- 200
 100200 <class 'str'> <class 'str'> <class 'str'>

Converting value into integer

In [11]:

```
1 a=int(input("Enter value of a:- "))
2 b=int(input("Enter value of b:- "))
3 c=a+b
4 print(c, " ", type(a), " ", type(b), " ", type(c))
```

Enter value of a:- 100
 Enter value of b:- 200
 300 <class 'int'> <class 'int'> <class 'int'>

In [12]:

```
1 a=int(input("Enter value of a:- "))
2 print(a)
```

Enter value of a:- 3.45

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-12-87c26932f0b0> in <module>()
----> 1 a=int(input("Enter value of a:- "))
      2 print(a)
```

ValueError: invalid literal for int() with base 10: '3.45'

Write a program to accept radius and calculate AOC

In [13]:

```
1 r=float(input("Enter value of Radius"))
2 AOC = 3.14*r*r
3 print("Area of Circle:- ",AOC)
```

Enter value of Radius10.50
Area of Circle:- 346.185

In [15]:

```
1 r=float(input("Enter value of Radius"))
2 print(r)
```

Enter value of Radius3
3.0

Output Through print() function

print() function use to print something in output screen

Format/Syntax of print():

- print(*objects ,[sep=' ' or<separator-string> end='\n'or<end-string>])

In [21]:

```
1 print(10)
2 print(121.5)
3 print("aman")
4 a=10
5 b=121.5
6 r="aman"
7 print(a)
8 print(b)
9 print(r)
10 print(10,a,121.5,b,"aman",c)
11 print(10,a,121.5,b,"aman",c,sep='#')
```

10
121.5
aman
10
121.5
aman
10 10 121.5 121.5 aman 300
10#10#121.5#121.5#aman#300

In [25]:

```
1 print(100,'aman',20.5,sep='_')
```

100_aman_20.5

In [23]:

```
1 print(sep='_ ',10,'aman',20.5)
```

File "<ipython-input-23-50883f6e06ec>", line 1

```
print(sep='_ ',10,'aman',20.5)
```

^

SyntaxError: positional argument follows keyword argument

In [28]:

```
1 print("i m python",end='____ ')
2 print("i m mouse",end='%')
3 print("i m cpu",end='#')
4 print("i m hemal")
5 print("i m surya")
```

i m python____i m mouse%i m cpu#i m hemal

i m surya

In [29]:

```
1 print("i m python",end='')
2 print("i m mouse")
```

i m pythoni m mouse

In [32]:

```
1 print("d\bhawal\b")
```

hawa

In [33]:

```
1 print("aman","hemal","suryansh",sep="\b")
```

amahemasuryansh

Write a program to obtain any three numbers and print sum and Average

In []:

```
1 a=int(input("Enter a value of A:- "))
2 b=int(input("Enter a value of B:- "))
3 c=int(input("Enter a value of C:- "))
4 sum=a+b+c
5 avg=sum/3
6 print("Sum :- ",sum)
7 print("Average :- ",avg)
```

Write a program to input a number and print its cube.

In []:

```
1 number=int(input("Enter Value for number"))
2 print(number**3)
```

Write a program to input a value in kilometres and convert it into miles(1km = 0.621371 miles)

In []:

```
1 KM=float(input("Enter value in KM:- "))
2 M=KM*0.621371
3 print(KM,"-> into Miles:- ",M)
```

Write a program to input two numbers and swap(interchange) them.

In []:

```
1 a=int(input("Enter Value of A:-"))
2 b=int(input("Enter Value of B:-"))
3 a,b=b,a
4 print("After Swap Values are:- ", "a:- ",a, " b:- ",b)
```

In []:

```
1 a=int(input("Enter Value of A:-"))
2 b=int(input("Enter Value of B:-"))
3 temp=a
4 a=b
5 b=temp
6 print("After Swap Values are:- ", "a:- ",a, " b:- ",b)
```

6. Output & Error Practice Questions

What will be the sizes of following constants:

In [44]:

```
1 print(len('\a'))
2 print(len("\a"))
3 print(len("Seema\s"))
4 print(len('\'))
5 print(len("it's"))
6 print("it's")
7 print(len("surya\
8 python"))
9 print(len(''surya
10 python'))
```

```
1
1
7
1
4
it's
11
12
```

Identify the types of following literals?

In []:

```
1 23.789
2 23789
3 'True'
4 True
5 "False"
6 0XFACE
7 0o789
8 None
```

Why is following code giving errors?

In [45]:

```
1 name="aman"
2 print("I m in XII class")
3     print("Yesterday only we received our XII result")
4     print("I got 100/100 in CS")
```

```
File "<ipython-input-45-cb90d8085bf2>", line 3
    print("Yesterday only we received our XII result")
    ^
```

IndentationError: unexpected indent

What will be the output produced by the following code?

In [46]:

```

1 name='simar'
2 age=17
3 print(name,"you are",17,"now but")
4 print("you will be",age+1,"next year")

```

simar ,you are 17 now but
you will be 18 next year

What will be the output of following code

In [47]:

```

1 x,y=2,6
2 x,y=y,x+2          #6 , 4
3 print(x,y)

```

6 4

Predict the output of following

In [49]:

```

1 x,y=7,2
2 x,y,x = x+1,y+3,x+10      #8 , 5 , 17
3 print(x,y)

```

17 5

Which of the following identifier names are invalid and why?

In []:

```

1 Serial_no.
2 1st_room
3 Hundred$
4 Total marks
5 total-marks
6 True
7 true

```

Write the corresponding Python assignment statements:

- Assign 10 to variable length and 20 to variable breadth
- Assign the average of values of variables length and breadth to a variable sum.

Write the output of the following:

In [50]:

```
1 num1=4
2 num2=num1+1
3 num1=2
4 print(num1,num2)
```

2 5

In [51]:

```
1 num1,num2=2,6
2 num1,num2=num2,num1+2      #6 , 4
3 print(num1,num2)
```

6 4

In [52]:

```
1 num1,num2=2,3
2 num3,num2=num1,num3+1
3 print(num1,num2,num3)
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-52-5a7e24d39309> in <module>()
      1 num1,num2=2,3
----> 2 num3,num2=num1,num3+1
      3 print(num1,num2,num3)
```

NameError: name 'num3' is not defined

End of Chapter

Next Chapter - Data Handling